

PROCESSO SELETIVO · ENGENHARIA

Desafio técnico

journey-core

Implemente o núcleo determinístico de uma jornada de saúde: consentimento, protocolo clínico data-driven, eventos de produto e elegibilidade de follow-up.

Nível	Prazo	Esforço esperado	Entrega
Pleno (backend)	3 dias corridos	12 a 16 horas	Repositório Git + README

Leia isto primeiro

Este desafio não pede chatbot, WhatsApp nem LLM. Queremos ver modelagem de domínio, arquitetura em camadas, regras de negócio testáveis e cuidado com dados pessoais (LGPD). Código que decide com inteligência artificial no lugar de regra determinística está fora do pedido.

1. Contexto

A AINA é um agente conversacional de saúde que acompanha pacientes no WhatsApp: onboarding, protocolos clínicos (como PHQ-9), plano de ação e follow-ups. No dia a dia de produto, a conversa usa linguagem natural — mas pontuação de protocolo, consentimento, trilha de eventos e decisão de “quem recebe um follow-up” precisam ser determinísticos, auditáveis e sem PII vazada.

Você vai construir uma API FastAPI (Python 3.12) chamada journey-core: o miolo que um agente usaria por baixo da conversa. Persistência pode ser in-memory. Integrações reais (Meta, Vertex AI, Firestore, BigQuery) estão fora de escopo.

2. O que entregar

- Repositório Git (público ou convite privado) com o código e os testes.
- README em português: como rodar, decisões de desenho, o que ficou de fora e o que você faria no 4º dia.
- Suíte pytest executável localmente, sem credenciais de nuvem.

Formato livre de pastas, desde que as camadas api → service → repository existam de forma reconhecível. O repositório pode ser um dict em memória; Firestore é bônus, não requisito.

3. Requisitos funcionais

3.1 Paciente e consentimento

POST /patients — cria paciente com phone, name, birth_date, sex e terms_accepted. Sem terms_accepted=true não é permitido iniciar protocolo nem jornada (responda 403 ou 409 com erro tipado).

Telefone é PII. Persista um identificador interno (patient_id) e um phone_hash (SHA-256 com salt configurável por variável de ambiente). O telefone em claro pode existir no cadastro operacional, mas nunca deve aparecer em eventos, logs ou respostas de erro.

3.2 Motor de protocolo data-driven

Carregue templates a partir de JSON (não hardcode a lista de perguntas no serviço). Inclua pelo menos o PHQ-9 (9 itens, Likert 0–3) com skip logic no estilo PHQ-2:

- Após as 2 primeiras perguntas, se a soma for menor que 3, encerre o bloco (end_block) e conclua o protocolo com pontuação parcial.
- Caso contrário, continue até o fim das 9 perguntas.
- O template é a única fonte das perguntas. Trocar o JSON não deve exigir if template_id == ... no serviço — só o interpretador genérico de skip logic.

Endpoints sugeridos (nomes equivalentes são aceitos):

- POST /patients/{id}/protocols — inicia sessão a partir de template_id.
- POST /protocol-sessions/{id}/answers — envia a próxima resposta; a API devolve a próxima pergunta ou o resultado final (score, status, ended_by_skip).

3.3 Jornada mínima e Event Store

Quando o protocolo concluir, crie uma jornada simples com status (em_andamento | concluída), objetivo (string) e tarefas (lista com id, título, status). Exponha um endpoint para marcar tarefa como concluída.

Toda transição relevante deve emitir um evento append-only (imutável: sem update/delete). Envelope mínimo:

event_id · occurred_at · event_name · patient_id_hash · properties
properties não pode conter telefone, nome ou data de nascimento.

Taxonomia mínima de eventos

event_name	Quando emitir
patient_created	Cadastro do paciente
terms_accepted	Aceite dos termos
protocol_started	Início da sessão de protocolo
protocol_completed	Fim do protocolo (inclusive por skip)
journey_created	Plano / jornada criado após o protocolo
task_completed	Tarefa marcada como concluída
followup_eligible	Motor de elegibilidade aprovou o disparo
followup_skipped	Motor recusou o disparo (com reason tipado)

GET /events?patient_id=... lista a trilha do paciente (use o id interno na query; a trilha persistida deve carregar o hash, não o telefone).

3.4 Motor de elegibilidade de follow-up

POST /followups/evaluate (ou disparo automático ao completar tarefa) aplica regras declarativas em JSON ou YAML, por exemplo:

- Paciente com termos aceitos.
- Protocolo concluído.
- Jornada em_andamento.
- Existe pelo menos uma tarefa em_andamento.
- Não disparar se já existe evento followup_eligible nas últimas 72 horas para o mesmo paciente.

Se elegível: emitir followup_eligible com template_key (ex.: checkin_adesao). Se não: emitir followup_skipped com reason tipado (missing_consent, no_active_task, cooldown, etc.). Não envie mensagem de verdade — só a decisão e o evento.

3.5 Qualidade esperada

- Type hints nas assinaturas públicas; Pydantic v2 nos contratos de API.
- Testes cobrindo: skip logic PHQ-2, recusa sem termos, PII ausente nos eventos, cooldown de 72h, jornada criada somente após protocolo.
- Nenhum segredo, chave ou dado de paciente real no repositório.

4. Especificação do PHQ-9

Use este enunciado (ou equivalente fiel). Escala para todas as perguntas: 0 Nenhuma vez · 1 Vários dias · 2 Mais da metade dos dias · 3 Quase todos os dias. Pergunta ao paciente: “Nas últimas duas semanas, com que frequência você foi incomodado por...”

#	Item
1	Pouco interesse ou prazer em fazer as coisas
2	Sentir-se para baixo, deprimido(a) ou sem esperança
3	Dificuldade para pegar no sono, continuar dormindo, ou dormir demais
4	Sentir-se cansado(a) ou com pouca energia
5	Falta de apetite ou comer demais
6	Sentir-se mal consigo mesmo(a) — ou que é um fracasso, ou que decepcionou a família ou a si mesmo(a)
7	Dificuldade para se concentrar nas coisas (ex.: ler ou ver televisão)
8	Lentidão para se mover ou falar, ou o oposto: inquietação a ponto de movimentar-se mais que o habitual
9	Pensamentos de que estaria melhor morto(a), ou de se ferir de alguma forma

Skip logic (PHQ-2)

Avaliar depois do item 2. Se $(\text{item1} + \text{item2}) < 3 \rightarrow \text{action} = \text{end_block}$, status concluído, ended_by_skip=true, score = soma parcial. Não invente índices compostos, “BLI” ou fórmulas clínicas além dessa soma.

5. Fora de escopo

- Integração WhatsApp / Meta Cloud API, Vertex AI, LangGraph, Langfuse.
- Dashboard, BigQuery, autenticação de usuário final, deploy em GCP.
- Interface gráfica.
- Fórmulas clínicas inventadas (bem-estar composto, “nível de confiança”, etc.).

6. Stack sugerida

Python 3.12 · FastAPI · Pydantic v2 · pytest · uv ou pip. Persistência: estruturas em memória (ou SQLite). Docker Compose é opcional. Bibliotecas extras são bem-vindas se justificadas no README.

7. Critério de “pronto”

Um revisor deve conseguir, em cerca de 15 minutos:

- Subir a API a partir do README.
- Criar paciente → responder PHQ até skip ou fim → ver a jornada criada.
- Avaliar follow-up duas vezes e observar o cooldown de 72h.
- Inspeccionar GET /events e confirmar ausência de PII.

8. Ritmo sugerido (3 dias)

Dia	Foco	Resultado visível
1	Modelos, paciente/consentimento, Event Store, testes de PII	Cadastro + trilha de eventos sem telefone
2	Motor de protocolo, skip logic, jornada	PHQ-9 via JSON; skip testado
3	Elegibilidade, cooldown, README, polimento	Regras declarativas e casos de borda

Como enviaremos o prazo
O relógio de 3 dias corridos começa na data/hora combinada no convite. Se um imprevisto impedir a entrega, avise antes do prazo — preferimos um recorte honesto a um repositório incompleto sem contexto.

Boa construção. Estamos avaliando julgamento de engenharia tanto quanto o código que compila.